# The Stabilization Protocol (Δ)

Governed Convergence Framework for the Adaptive Autonomous Intelligence System (AAIS)

---

| | |
|---|---|
| **Document ID:** | **AAIS-SP-Δ-001** |
| **Version:** | **1.0.0** |
| **Classification:** | **Publication-Ready / Governance Artifact** |
| **Companion Document:** | **AAIS-VB-Λ-001 — The Voss Binding** |
| **Author:** | **Jon Halstead, Cognitive Systems Designer** |
| **Date:** | **May 2, 2026** |
| **Repository:** | **Project-Infinity** |
| **Status:** | **RATIFIED** |

---

# §0 — Preamble

*This document constitutes The Stabilization Protocol (Δ) — the binding convergence and operational discipline framework for the Adaptive Autonomous Intelligence System (AAIS). Where The Voss Binding (Λ) defines what must be true, The Stabilization Protocol defines how the system arrives at and maintains that truth. Δ governs the transitions between operational states, the resolution of instability, and the restoration of invariant compliance. No convergence procedure, recovery action, or state transition within AAIS operates outside the authority of this protocol. This version includes the Implementation Architecture — the concrete*

*enforcement mechanisms, state machine engines, and convergence automation that translate these protocols from procedure into structural enforcement.*

---

# §1 — Definitions

**Δ (Delta / Stabilization):** The complete set of convergence protocols, state transitions, and recovery procedures governing AAIS operational discipline.

**Stability:** The condition where all active modules, lanes, and agents operate within their declared Λ-compliant contracts with zero detected drift.

**Convergence:** The process by which a system moves from an unstable or partially-compliant state toward full Λ compliance.

**Perturbation:** Any event that displaces the system from its stable operating point — including new deployments, configuration changes, external inputs, or detected anomalies.

**Recovery Corridor:** The bounded operational space within which convergence procedures execute — constrained by Λ laws at all times.

**Quiescence:** The terminal stable state where all modules report nominal, all drift metrics are zero, and no convergence procedures are active.

**Stabilization Epoch:** A bounded time window during which convergence procedures execute and the system transitions toward quiescence.

**Cascade Boundary:** The architectural limit preventing a perturbation in one module or lane from propagating to adjacent scopes.

**Health Vector:** The composite real-time metric representing system stability across all active modules, lanes, and agents.

**Operator Checkpoint:** A mandatory pause point during stabilization where the Operator reviews system state before convergence continues.

---

# §2 — The Five Stabilization Axioms

**Δ.1 — Bounded Convergence.** Every stabilization procedure must converge within a declared time boundary. Unbounded convergence is a violation — if convergence cannot complete within the epoch, the system fails closed and surfaces to the Operator.

**Δ.2 — Monotonic Progress.** During convergence, the system's Health Vector must improve monotonically. Any regression triggers immediate halt and Operator escalation. The system never gets worse during recovery.

**Δ.3 — Cascade Isolation.** A perturbation in any module, lane, or agent must not propagate beyond its declared cascade boundary. Isolation is structural, not aspirational — enforced by architecture, not policy.

**Δ.4 — Operator-Gated Transitions.** Critical state transitions during stabilization require explicit Operator approval at defined checkpoints. The system proposes; the Operator decides.

**Δ.5 — Reversibility.** Every convergence action must be reversible within the current stabilization epoch. If an action cannot be undone, it requires Operator authorization before execution.

---

# §3 — System States and Transitions

## §3.1 — State Definitions

**NOMINAL (Green):** Full Λ compliance, zero drift, all contracts satisfied. Normal operation.

**PERTURBED (Amber):** A perturbation has been detected. Drift metrics are non-zero but within recovery tolerance. Stabilization epoch initiated.

**CONVERGING (Blue):** Active stabilization procedures executing within recovery corridor. Health Vector improving monotonically.

**DEGRADED (Orange):** Convergence stalled or Health Vector regression detected. Operator escalation active. Partial capability may continue under Operator authorization.

**HALTED (Red):** Fail-closed state. Critical invariant violation or convergence failure. All affected scopes suspended. Awaiting Operator intervention.

**QUIESCENT (White):** Post-convergence terminal state. All metrics nominal. System returns to NOMINAL after Operator confirmation.

## §3.2 — State Transition Table

| From State | To State | Trigger Condition | Authorization |
|---|---|---|---|
| NOMINAL | PERTURBED | Perturbation detected; drift metrics become non-zero | Automatic |
| PERTURBED | CONVERGING | Convergence plan | Operator |

| From State | To State | Trigger Condition | Authorization |
|---|---|---|---|
| | | approved at Operator Checkpoint | |
| CONVERGING | QUIESCENT | All convergence procedures complete; Health Vector nominal | Automatic |
| CONVERGING | DEGRADED | Health Vector regression or convergence stall detected | Automatic |
| DEGRADED | CONVERGING | Operator authorizes revised convergence plan | Operator |
| DEGRADED | HALTED | Critical invariant violation or epoch timeout | Automatic |
| HALTED | CONVERGING | Operator diagnoses root cause and authorizes recovery | Operator |
| QUIESCENT | NOMINAL | Operator confirms post-convergence verification | Operator |
| Any State | HALTED | Critical Λ-invariant violation detected | Automatic |

---

# §4 — State Machine Implementation

The state machine is not a diagram — it is a runtime engine. Every state transition is enforced programmatically, logged immutably, and gated by the authorization rules declared in §3.

# §4.1 — State Machine Engine (SME)

The State Machine Engine is the runtime component that manages all system state transitions. It maintains a single authoritative state register for each scope — module, lane, agent, and system-wide. There is no secondary source of state truth. The SME is the register.

State transitions are atomic. The system is never between states. A transition either completes fully or does not occur — there is no partial transition, no intermediate limbo, no "in progress" state change. The SME validates every transition request against the State Transition Table (§3.2) before execution. Invalid transition requests — such as NOMINAL → QUIESCENT without intermediate states — are rejected, logged as governance violations, and surfaced to the Operator.

## Transition Processing Pipeline

| Step | Phase | Description |
|---|---|---|
| 1 | **TRANSITION REQUEST** | A state change is requested by the GRE, Drift Monitor, Operator, or an active convergence procedure. The request includes: requester identity, target state, justification, and authorization level. |
| 2 | **VALIDATION** | SME checks the request against the State Transition Table — is this transition legal from the current state? If not, the request is rejected and logged. |
| 3 | **AUTHORIZATION CHECK** | SME verifies the required authorization level: Automatic (no human gate), Operator Checkpoint (review required), or Operator Explicit (direct Operator command required). |
| 4 | **PRE-TRANSITION** | Current state is captured |

| Step | Phase | Description |
|---|---|---|
| | **SNAPSHOT** | and archived before any change occurs. The snapshot includes: state register value, Health Vector, active convergence procedures, and timestamp. |
| 5 | **STATE WRITE** | The new state is written to the authoritative state register. This is the atomic commit point — once written, the transition is complete. |
| 6 | **POST-TRANSITION VERIFICATION** | SME confirms the state register reflects the intended state. If verification fails, the system fails closed and the pre-transition snapshot is used for recovery. |
| 7 | **EVENT EMISSION** | State change event emitted to: Governance Surface, Audit Trail, Health Vector Engine, and any active convergence procedures. |
| 8 | **NOTIFICATION** | If the transition requires Operator awareness, notification is dispatched via the Governance Surface with full transition context. |

## §4.2 — Scope Hierarchy

State is managed at four hierarchical levels. State at each level is derived from the scopes beneath it.

**Module Scope:** Individual module state. Managed by the module's circuit breaker and GRE pipeline. The most granular level of state management.

**Lane Scope:** Aggregate state of all modules in a lane. A lane is NOMINAL only when ALL its modules are NOMINAL. A lane transitions to PERTURBED when ANY module drifts. The lane state is the worst state of its constituent modules.

**Agent Scope:** Agent-level state including identity integrity and corrigibility status. Agent state is independent of lane state — an agent may be NOMINAL while the lane it operates within is PERTURBED, provided the agent's own contracts remain satisfied.

**System Scope:** Global state aggregated from all lanes and agents. The system state is the WORST state of any active scope — if one lane is HALTED, the system reports HALTED for that scope boundary. The system is only as healthy as its most compromised scope.

## §4.3 — Transition Guards

Every transition passes through a series of guards before execution. Guards are structural enforcement — they cannot be bypassed, disabled, or overridden at runtime.

| Guard | Function | Applies To |
|---|---|---|
| **Authorization Guard** | Validates that the required authorization (Automatic, Operator Checkpoint, Operator Explicit) has been obtained before allowing the transition. | All transitions |
| **Health Vector Guard** | For transitions TO NOMINAL or QUIESCENT, validates that the Health Vector meets the required threshold. A system cannot declare itself healthy without proof. | → NOMINAL, → QUIESCENT |
| **Cascade Guard** | For transitions that could affect adjacent scopes, validates that cascade boundaries are maintained per Δ.3. No transition may propagate instability. | Cross-scope transitions |
| **Epoch Guard** | For transitions during active stabilization, validates that the transition is consistent with the current convergence plan | Transitions during convergence |

| Guard | Function | Applies To |
|---|---|---|
| | and does not violate the epoch boundary. | |
| **Reversal Guard** | Validates that every transition has a defined reversal path per Δ.5. If no reversal path exists, the transition requires explicit Operator authorization. | All transitions |

---

# §5 — Convergence Procedures

The standard convergence protocol consists of eight sequential phases. Each phase must complete successfully before the next begins. Failure at any phase triggers the defined failure mode.

**Step 1 — DETECT.** Perturbation identified via continuous Λ-invariant monitoring across all active modules, lanes, and agents.
*Failure Mode:* If detection systems are themselves compromised, the system transitions immediately to HALTED and alerts the Operator via out-of-band notification.

**Step 2 — ISOLATE.** Affected scope bounded at cascade boundary. Adjacent scopes notified but not impacted.
*Failure Mode:* If isolation fails and perturbation propagates beyond the cascade boundary, all affected scopes transition to HALTED per Axiom Δ.3.

**Step 3 — DIAGNOSE.** Root cause analysis within the isolated scope. Drift category identified: Behavioral, Schema, Identity, or Temporal — as defined in Λ §4.
*Failure Mode:* If root cause cannot be determined within the diagnostic time allocation, the system escalates to DEGRADED and requests Operator-directed diagnosis.

**Step 4 — PROPOSE.** Convergence plan generated with estimated epoch duration, affected modules, and reversibility assessment for every planned action.

*Failure Mode:* If no viable convergence plan can be generated, the system remains in PERTURBED and escalates to the Operator with full diagnostic data.

**Step 5 — CHECKPOINT.** Operator reviews convergence plan at the mandatory checkpoint. Approval required to proceed with execution.

*Failure Mode:* If Operator rejects the plan, the system returns to Step 4 (PROPOSE) with Operator-supplied constraints. If Operator is unreachable, the system holds in PERTURBED until contact is established.

**Step 6 — EXECUTE.** Convergence actions applied within recovery corridor. Health Vector monitored continuously. Monotonic progress verified per Axiom Δ.2.

*Failure Mode:* If Health Vector regresses at any point during execution, all actions are immediately reversed (Axiom Δ.5) and the system transitions to DEGRADED.

**Step 7 — VERIFY.** Post-convergence invariant verification across all affected contracts. Every module within the affected scope re-validated against its Λ-compliant contract.

*Failure Mode:* If verification detects residual drift, the system returns to Step 3 (DIAGNOSE) for a new convergence cycle within the remaining epoch.

**Step 8 — QUIESCE.** System transitions to QUIESCENT state. All metrics confirmed nominal. Operator confirms return to NOMINAL.

*Failure Mode:* If quiescence cannot be achieved within the stabilization epoch, the system transitions to DEGRADED and initiates a new epoch with Operator authorization.

---

# §6 — Convergence Engine Implementation

## §6.1 — Convergence Orchestrator

The Convergence Orchestrator manages the execution of convergence procedures defined in §5. It is the runtime authority for all active stabilization epochs — no convergence action executes outside its governance.

The Orchestrator maintains a Convergence Plan object for each active stabilization epoch. The plan contains:

| Field | Description |
|---|---|
| **Affected Scopes** | All modules, lanes, and agents within the convergence boundary |
| **Proposed Actions** | Ordered list of convergence actions with target scope and action type |
| **Estimated Duration** | Epoch time boundary — the maximum duration before fail-closed triggers |
| **Reversibility Assessment** | Per-action reversibility classification (reversible, conditionally reversible, irreversible) |
| **Operator Approvals** | Checkpoint approval status and any Operator-supplied constraints or modifications |

The Orchestrator enforces Bounded Convergence (Δ.1) by maintaining an epoch timer. If convergence does not complete within the declared boundary, the Orchestrator triggers fail-closed — no extensions, no renegotiation, no silent overruns. The Orchestrator enforces Monotonic Progress (Δ.2) by sampling the Health Vector at configurable intervals and halting immediately on any regression.

## §6.2 — Convergence Action Queue

All convergence actions are queued in a governed FIFO queue. Actions execute in declared order — no reordering, no priority overrides, no queue jumping without Operator authorization.

| Field | Description |
|---|---|
| **action_id** | Unique identifier for this convergence action |
| **target_scope** | Module, lane, or agent targeted by this action |
| **action_type** | Recovery pattern type (Hot Restart, Warm Rollback, Cold Isolation, Surgical Correction, Full Reconvergence) |
| **estimated_duration** | Projected execution time for this action |
| **reversibility** | Boolean — true if the action can be undone within the epoch |
| **operator_authorization_required** | Boolean — true if the action requires explicit Operator approval before execution |
| **rollback_procedure** | Reference to the defined rollback steps if this action must be reversed |

Actions execute sequentially within a scope — no parallel convergence actions on the same scope. Cross-scope convergence actions require cascade boundary verification before execution per Δ.3.

## §6.3 — Rollback Manager

The Rollback Manager maintains a stack of reversible actions for each active convergence epoch. The stack is the system's memory of what it has done — and the blueprint for undoing it.

- If Monotonic Progress is violated (Health Vector regression), the Rollback Manager unwinds actions in reverse order — last action first, working backward through the stack.

- Each rollback action re-validates the Health Vector after execution to confirm improvement. Rollback that does not improve the Health Vector is itself a failure.
- If rollback causes further regression, the system fails closed immediately and escalates to the Operator. A system that cannot undo its own recovery is a system that has lost control.
- The Rollback Manager's stack is preserved in the audit log for post-incident analysis. No stack entry is discarded — even after successful convergence, the history of what was attempted and what was reversed is permanent record.

## §6.4 — Operator Checkpoint Manager

The Operator Checkpoint Manager governs the checkpoint gates defined in the convergence procedure (§5, Step 5). It is the structural interface between autonomous convergence and human authority.

When a checkpoint is reached, the Operator Checkpoint Manager:

1. Pauses convergence execution — no further actions proceed.
2. Generates a checkpoint report containing: current system state, actions taken so far, Health Vector trajectory, proposed next actions, and reversibility assessment.
3. Dispatches the report to the Governance Surface for Operator review.
4. Waits for Operator response. The system does not proceed without authorization.

Operator response options:

| Response | Effect |
| --- | --- |
| **APPROVE** | Continue with the proposed convergence plan as submitted. |
| **MODIFY** | Operator adjusts the convergence plan — modified plan replaces the original in the Orchestrator. |
| **REJECT** | Halt convergence. Hold current state. The system does not attempt further recovery without a new plan. |
| **ESCALATE** | Operator takes manual control. All automated convergence is suspended. The Operator directs recovery directly. |

**Checkpoint Timeout:** If the Operator does not respond within a configurable timeout, the system holds in its current state indefinitely. It never proceeds without authorization. Timeout is not implicit approval — it is a pause that persists until a human decides.

---

# §7 — Health Vector Specification

## §7.1 — Component Dimensions

**Module Health:** Contract compliance percentage across all active modules, measured as the ratio of satisfied invariants to total declared invariants.

**Lane Health:** Pipeline integrity and throughput within governance bounds, measured as the ratio of compliant data flows to total active flows.

**Agent Health:** Identity integrity, drift metrics, and corrigibility verification across all active agents, measured against their declared identity contracts.

**System Health:** Aggregate weighted score across all dimensions, computed as a configurable weighted mean with Operator-defined dimension weights.

## §7.2 — Threshold Categories

| Threshold | Score Range | Classification | System Response |
|---|---|---|---|
| **Optimal** | 1.0 | Perfect Λ compliance across all dimensions | NOMINAL operation; no action required |
| **Nominal** | ≥ 0.95 | Within acceptable operational variance | NOMINAL operation; standard monitoring |
| **Warning** | 0.85 – 0.94 | Elevated monitoring threshold | Increased monitoring frequency; no automated action |
| **Critical** | 0.70 – 0.84 | Stabilization required | Stabilization epoch triggered automatically |
| **Emergency** | < 0.70 | System integrity at risk | Immediate fail-closed; Operator escalation |

---

# §8 — Health Vector Engine Implementation

## §8.1 — Health Vector Computation Engine

The Health Vector Engine (HVE) continuously computes the composite health metric from all active scopes. It is not a polling mechanism — it is a governed computation cycle that runs on a configurable interval: every 1 second for critical systems, every 5 seconds for standard operation.

The HVE collects dimensional scores from each scope — Module Health, Lane Health, Agent Health — and computes the System Health as a weighted aggregate:

**System_Health = min($w_m$ × Module_Health_avg, $w_l$ × Lane_Health_min, $w_a$ × Agent_Health_min)**

Where weights ($w_m$, $w_l$, $w_a$) are configurable but must sum to 1.0. The HVE uses a minimum function for lane and agent health because the system is only as healthy as its weakest governed scope. An average would mask a failing scope behind healthy neighbors — the minimum function refuses that deception.

## §8.2 — Threshold Engine

The Threshold Engine evaluates the computed Health Vector against the five threshold categories defined in §7.2 (Optimal, Nominal, Warning, Critical, Emergency). Threshold crossings are treated as state-changing events — they trigger the SME transition pipeline.

Hysteresis is applied to prevent threshold oscillation: a threshold crossing must persist for a configurable number of computation cycles (default: 3) before triggering action. A system that oscillates across a threshold is unstable — but it should not trigger cascading responses from momentary fluctuation.

The Threshold Engine emits threshold events to:

| Recipient | Receives Events | Purpose |
|---|---|---|
| **Governance Surface** | All threshold crossings | Operator visibility into Health Vector movement |
| **State Machine Engine** | Critical and Emergency crossings | Trigger state transitions per §3.2 |

| Recipient | Receives Events | Purpose |
|---|---|---|
| **Convergence Orchestrator** | Critical crossings | Initiate stabilization epoch |
| **Kill Switch** | Emergency crossings | Immediate fail-closed across all scopes |

## §8.3 — Health Vector History

The HVE maintains a rolling history buffer of Health Vector values for trend analysis. The history window is configurable, with a default of 24 hours at 1-second resolution.

Trend analysis capabilities:

- **Gradual Degradation:** Detection of slow drift toward threshold boundaries — the system recognizes a slope before it becomes a cliff.
- **Oscillation:** Detection of instability in specific dimensions — a Health Vector that bounces indicates an unresolved perturbation.
- **Sudden Drops:** Detection of perturbation signatures — a sharp Health Vector decline is the fingerprint of a discrete event.

History data is included in convergence checkpoint reports (§6.4) and post-stabilization audit reports (§11). The Operator sees not just where the system is, but where it has been and where it is trending.

---

# §9 — Recovery Patterns

| Pattern | Trigger Condition | Scope | Est. Duration | Operator Auth. | Reversible |
|---|---|---|---|---|---|
| **Hot Restart** | Transient module | Single module | Seconds | No | Yes |

| Pattern | Trigger Condition | Scope | Est. Duration | Operator Auth. | Reversible |
|---|---|---|---|---|---|
| | fault; Health Vector ≥ 0.85 | within cascade boundary | | | |
| **Warm Rollback** | Persistent drift after Hot Restart; known-good state available | Single module; lane paused during rollback | Minutes | No | Yes |
| **Cold Isolation** | Drift source unidentified; cascade risk elevated | Full scope isolation; no active processing | Minutes to hours | Yes | Yes |
| **Surgical Correction** | Operator-identified root cause requiring targeted modification | Specific module state or configuration | Variable | Yes | Conditional |
| **Full Reconvergence** | Major perturbation, deployment, or multi-scope drift | System-wide; all modules re-verified | Hours | Yes | Yes |

**Note on Surgical Correction:** Reversibility is conditional — if the targeted modification is irreversible by nature, Operator authorization must include explicit acknowledgment of irreversibility per Axiom Δ.5.

---

# §10 — Recovery Pattern Implementation

## §10.1 — Recovery Executor

The Recovery Executor is a specialized component of the Convergence Orchestrator that implements the five recovery patterns defined in §9. Each

pattern has a corresponding executor class with four mandatory phases: precondition checks, execution logic, post-condition verification, and rollback procedure.

The Recovery Executor validates preconditions before executing any pattern. If preconditions fail, the pattern is not attempted and the Operator is notified with the reason for precondition failure and the recommended alternative. A recovery that cannot verify its own starting conditions is not ready to recover.

## §10.2 — Pattern Selection Engine

When a convergence plan is generated (Step 4 of §5), the Pattern Selection Engine recommends the appropriate recovery pattern based on:

| Selection Factor | Influence on Pattern Selection |
|---|---|
| **Perturbation Type** | Behavioral drift favors Hot Restart; Identity drift requires Cold Isolation minimum |
| **Affected Scope Size** | Single-module scope favors targeted patterns; multi-scope favors Full Reconvergence |
| **Drift Magnitude** | Minor drift permits least-invasive patterns; severe drift escalates to comprehensive recovery |
| **Time Constraints** | Tight epoch boundaries favor faster patterns (Hot Restart, Warm Rollback) |
| **Reversibility Requirements** | High reversibility requirements exclude Surgical Correction unless Operator authorizes |

**Selection Priority:** Hot Restart (fastest, least disruptive) → Warm Rollback → Cold Isolation → Surgical Correction → Full Reconvergence (most comprehensive). The engine always recommends the LEAST invasive pattern that can resolve the perturbation within the epoch boundary. The Operator may override the recommendation at the checkpoint.

## §10.3 — Pattern Execution Monitoring

During recovery pattern execution, the Recovery Executor reports continuously to the Governance Surface:

- **Current Phase:** Which phase of the recovery pattern is executing (precondition, execution, verification, or rollback)
- **Elapsed Time:** Time consumed against the epoch boundary
- **Health Vector Delta:** Real-time Health Vector change since recovery began
- **Actions Completed:** Count and summary of convergence actions successfully applied
- **Actions Remaining:** Count and estimated duration of remaining actions in the convergence plan

All monitoring data flows through the Governance Surface in real time. If any pattern execution causes Health Vector regression, the executor halts immediately per Δ.2 — the Rollback Manager takes over.

---

# §11 — Stabilization Logging and Audit

All stabilization events must be logged with the following mandatory fields:

- **Timestamp:** UTC time of event occurrence at millisecond precision
- **Affected Scope:** Module, lane, agent, or system-level identifier
- **Perturbation Type:** Classification per Λ §4 drift categories (Behavioral, Schema, Identity, Temporal)
- **Convergence Procedure:** Step identifier and action description

- **Health Vector Deltas:** Pre-action and post-action Health Vector values across all dimensions
- **Operator Actions:** All Operator decisions, approvals, rejections, and directives
- **Outcome:** Terminal state and disposition of the convergence procedure

Logs are immutable during the stabilization epoch. No log entry may be modified, deleted, or overwritten while any convergence procedure is active.

A post-stabilization audit report is generated automatically upon transition to QUIESCENT and surfaced to the Operator for review. The audit trail must satisfy Λ.2 (Auditability) — every convergence action must be traceable to its originating trigger with an unbroken chain of evidence.

---

# §12 — Integration with The Voss Binding (Λ)

The relationship between Δ and Λ is hierarchical and non-negotiable:

- Δ operates within the boundaries set by Λ — it never overrides, suspends, or circumvents Λ laws under any operational condition.
- Δ provides the operational procedures that maintain Λ compliance. Where Λ declares the invariant, Δ enforces the discipline.
- When Δ and Λ conflict, Λ prevails absolutely, per the principle of Governance Supremacy (Λ.7).

- Δ convergence procedures must themselves be Λ-compliant — the recovery process is governed by the same laws it seeks to restore. There is no exception state.

## §12.1 — Axiom Cross-Reference

| Δ Axiom | Supporting Λ Law(s) | Relationship |
|---|---|---|
| Δ.1 — Bounded Convergence | Λ.3 (Temporal Integrity) | Convergence time boundaries derive from Λ temporal contracts |
| Δ.2 — Monotonic Progress | Λ.2 (Auditability), Λ.5 (Behavioral Fidelity) | Progress must be auditable; regression violates behavioral contracts |
| Δ.3 — Cascade Isolation | Λ.4 (Scope Sovereignty), Λ.6 (Identity Integrity) | Scope boundaries are inviolable; identity must not bleed across scopes |
| Δ.4 — Operator-Gated Transitions | Λ.1 (Operator Supremacy), Λ.7 (Governance Supremacy) | The Operator retains ultimate authority over all system transitions |
| Δ.5 — Reversibility | Λ.1 (Operator Supremacy), Λ.8 (Corrigibility) | Reversibility ensures the Operator can always correct the system |

---

# §13 — Deployment and Integration Requirements

## §13.1 — Stabilization System Deployment Checklist

Before the stabilization system is operational, the following must be verified. No component may enter NOMINAL state until all checklist items are confirmed.

| # | Component | Verification Requirement |
|---|---|---|
| 1 | State Machine Engine | Initialized with all scopes registered and in NOMINAL state |
| 2 | Convergence Orchestrator | Operational with epoch timer calibrated to scope-appropriate boundaries |
| 3 | Health Vector Engine | Computing and emitting scores on schedule; all dimensional inputs verified |
| 4 | Threshold Engine | Configured with appropriate threshold values and hysteresis cycles |
| 5 | Rollback Manager | Initialized with empty stacks for all scopes; stack write verified |
| 6 | Operator Checkpoint Manager | Connected to Governance Surface; report generation and dispatch confirmed |
| 7 | Recovery Executor | All five pattern executors loaded; precondition validators operational |
| 8 | Pattern Selection Engine | Calibrated with scope-appropriate parameters and selection priorities |
| 9 | Audit Trail | Connection verified; chain integrity confirmed; immutability tested |
| 10 | GRE Integration | Drift scores flowing from GRE to HVE; circuit breaker events reaching SME |
| 11 | Kill Switch | Tested and confirmed operational across all scopes |
| 12 | Operator Authentication | Verified for all checkpoint and escalation paths; timeout behavior confirmed |

## §13.2 — Integration Interfaces

The stabilization system exposes and consumes the following interfaces. Every governed component must implement the required interfaces — a component without these interfaces cannot participate in governed convergence.

| Interface | Direction | Methods | Purpose |
|---|---|---|---|
| **StateAware** | Consumed from all governed components | getCurrentState(), requestTransition(), getTransitionHistory() | Enables the SME to manage and query component state |
| **Convergeable** | Implemented by all recoverable components | prepareForConvergence(), executeRecoveryAction(), verifyPostRecovery(), rollback() | Enables the Recovery Executor to manage component recovery |
| **HealthContributor** | Implemented by all monitored components | getDimensionalHealth(), getDriftScores(), getContractCompliancePercentage() | Feeds dimensional health data to the HVE |
| **OperatorGated** | Implemented by checkpoint-capable components | generateCheckpointReport(), awaitOperatorDecision(), applyOperatorModification() | Enables the Operator Checkpoint Manager to gate component transitions |

## §13.3 — GRE Integration

The Stabilization Protocol integrates with the Governance Runtime Engine (GRE) defined in The Voss Binding (Λ). This integration is structural — the GRE and the stabilization system are not loosely coupled services but tightly governed components of a single governance architecture.

- The GRE's Drift Measurement stage feeds drift scores directly to the Health Vector Engine. Drift data flows in real time — the HVE does not poll for drift, it receives it.
- The GRE's Circuit Breaker state changes trigger SME transition requests. When a circuit breaker opens, the SME receives a transition request for the affected scope.
- The GRE's Audit Emission feeds the stabilization audit trail. All governance events from module execution are captured in the same audit chain as stabilization events — one unbroken record.
- The Convergence Orchestrator can instruct the GRE to pause, resume, or reconfigure module execution during recovery. The GRE accepts these instructions as governed commands, not suggestions.
- The Kill Switch simultaneously triggers all GRE circuit breakers to OPEN state. When the Kill Switch fires, every module in every lane halts. There is no selective Kill Switch — it is total and immediate.

---

# §14 — Amendment Protocol

The Stabilization Protocol may only be amended through the following procedure:

5. **Formal Written Proposal:** The proposed amendment must be documented in writing with precise identification of the protocol sections affected, the proposed changes, and the rationale.
6. **Impact Analysis:** A formal assessment of the amendment's impact on all active convergence procedures, recovery patterns, and state transitions must be completed and documented.

7. **Λ Compatibility Verification:** The proposed amendment must be verified for full compatibility with the current version of The Voss Binding (Λ). No amendment to Δ may introduce a conflict with Λ.
8. **Operator Approval:** The Operator must review and explicitly approve the amendment. No amendment takes effect without Operator ratification.
9. **Version Increment:** Upon approval, the document version is incremented and the amendment is recorded in the document history with date, author, and scope of change.

No runtime modification of Δ protocols is permitted. The Stabilization Protocol is a static governance artifact — it is read at system initialization and may only be changed through the formal amendment process defined above.

---

# §15 — Ratification

This document — **The Stabilization Protocol (Δ), Version 1.0.0, dated May 2, 2026** — is hereby ratified as the binding convergence, recovery, and state transition framework for the Adaptive Autonomous Intelligence System (AAIS).

All convergence procedures, recovery patterns, state transitions, and stabilization epochs operating within AAIS are bound by these protocols without exception. No module, lane, agent, or system process may execute a convergence action outside the authority of this document. All implementation components — the State Machine Engine, Convergence Orchestrator, Health Vector Engine, Threshold Engine, Rollback Manager, Recovery Executor, Pattern Selection Engine, and Operator Checkpoint

Manager — are binding alongside the stabilization protocols they enforce. The implementation is not separate from the governance — it is the governance made structural.

This protocol operates as a companion to and under the authority of The Voss Binding (Λ), Document ID AAIS-VB-Λ-001. Where Λ is the constitution, Δ is the operational discipline. Together, they form the complete governance framework for AAIS.

So ratified.

---

**Jon Halstead**

Architect and Operator

Cognitive Systems Designer — Project-Infinity

May 2, 2026

---